POLITECNICO
MILANO 1863

AN2DL - First Challenge Report
I Neuralini

Tommaso Alongi, Federico Carugati, Francesco Paolini, Alessandro Romani

tommasoalongipolimi, fedepolitecnico, francescopaolini, aleromani4

306455, 303715, 300560, 303256

November 17, 2025

1 Introduction

The following project is primarily focused on training and building a *Recurrent Neural Network (RNN)* model to classify pain levels in pirate-themed time-series data.

The core problem is *multi-class classification* of pirate pain levels: **no pain, low pain, high pain**.

The key challenge of the project relies on effectively processing the *time-series nature* of the data and integrating it with the *static features* to predict a single pain label per pirate sample.

1.1 Project's Goals

The primary objects of this projects are:

- Develop a robust and accurate classification model for pirate pain, allowing for generalization.
- Achieve high **F1-score** for unseen test data, indicating a good balance between precision and recall.

1.2 The Team's Approach

The workflow consisted of three main phases:

1. Data preprocessing through min–max normalization, user-wise splitting, and windowing to prevent data leakage;
2. Baseline development using several recurrent architectures (RNN, LSTM, GRU, both uni- and bidirectional);
3. Performance refinement via k-shuffle-split cross-validation, grid-search hyperparameter tuning, and the exploration of additional advanced architectures.

2 Problem Analysis

2.1 Dataset Characteristics Overview

The dataset is organized as a collection of time-series samples, each consisting of 160 time steps associated with unique pirate entries (sample indices). The prediction task is framed as a multi-class classification problem involving three pain levels: *no pain*, *low pain*, and *high pain*.

The dataset comprises both categorical and dynamic information:

- **Time-Series:** Joint measurements from `joint_00` to `joint_30`, represented as numerical sensor readings.
- **Survey Data:** Pain survey responses `survey_1` to `survey_4`, encoded as integer values.
- **Categorical/Static Features:** Pirate attributes such as the number of legs, hands, and eyes.

A pronounced imbalance among the three classes was observed, leading to several considerations later discussed in the subsequent sections. Additionally, `joint_30` was excluded from the analysis, as it exhibited a constant value across all time steps and was therefore deemed uninformative.

2.2 Main Challenges

The temporal nature of the data directly influenced the architectural choices for the models employed in the project.

It was of paramount importance to determine the optimal temporal parameters for the neural networks to effectively capture the information embedded in the time series data.

Moreover, the substantial class imbalance required the use of robust training strategies, such as weighted cross-entropy loss and data augmentation techniques, to mitigate bias and improve model generalization.

2.3 Initial Assumptions

Given the temporal relevance of the dataset, we explored the impact of varying time-window sizes in order to control the extent of historical information accessible to the recurrent models. All provided features were assumed to carry meaningful information.

3 Method

In this section, we describe the methodologies, architectures, and algorithms adopted throughout the project, outlining the main steps that guided our workflow.

Given the strong dataset imbalance, we implemented a Weighted Cross-Entropy Loss to assign higher penalties to errors on minority classes and mitigate bias. The method assigns larger penalty weights to misclassifications involving minority classes (`low_pain` and `high_pain`) and smaller weights to errors on the majority class (`no_pain`). The Weighted Cross-Entropy Loss ($\mathcal{L}_{WCE}$) incorporates class-specific weights w_i 2, set inversely to class frequencies, to adjust the standard Cross-Entropy formulation.

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N} y_i \log(\hat{y}_i) \tag{1}$$

$$\mathcal{L}_{WCE} = -\frac{1}{N}\sum_{i=1}^{N} w_i\, y_i \log(\hat{y}_i) \tag{2}$$

We also investigated data augmentation techniques, such as Time Warping and SMOTE, to generate plausible synthetic time-series samples.

Further analysis revealed that certain subsequences—typically central ones—were significantly more reliable. This led to a manual attention-like strategy that selected the most informative subsequence, improving prediction performance.

To generalize this idea, we later introduced an automatic attention mechanism that learns relevance weights across subsequences.

Finally, an extensive hyperparameter tuning process, based on grid search and k-fold cross-validation, enabled us to identify the architecture achieving the best validation performance.

4 Experiments

This section presents the experimental evaluation, reporting results from recurrent baselines to progressively enhanced models incorporating cross-validation, hyperparameter tuning, data augmentation, weighted losses and attention mechanisms. All experiments were conducted with fixed random seeds for reproducibility and the macro-averaged F1-score was used as the primary metric.

4.1 Baseline Recurrent Models

We first established baseline performance using simple recurrent architectures (`RNN`, `LSTM`, `GRU`) with default hyperparameters (hidden size = 128, 2 recurrent layers, dropout = 0.2, learning rate = 1e-3, no regularization). Table 1 summarizes validation performance.

Table 1: Baseline performance.

Model	Accuracy	Precision	F1-score
RNN	0.9402	0.9392	0.9389
LSTM	0.9451	0.9437	0.9428
bi-LSTM	0.9483	0.9476	0.9472
GRU	0.9507	0.9512	**0.9494**
bi-GRU	0.9442	0.9434	0.9434

The GRU-based model outperformed other RNNs, validating its suitability for the considered time-series task. However, the LSTM model was selected as a starting point for further exploration due to its better performance on following tests.

It should be noted that all implemented baseline models exhibited a pronounced tendency to overfit. Therefore, building upon the baseline experiments, hyperparameter tuning was performed via grid search to address overfitting by identifying configurations with lower complexity and enhanced regularization (including higher dropout rates). This effort successfully met expectations, resulting in superior performance from the refined LSTM architecture (Tuned LSTM) relative to its initial configuration.

4.2 Further implementations

To address class imbalance and improve temporal focus, we jointly tested the Weighted Cross-Entropy Loss and an attention mechanism, observing only marginal performance gains.

To achieve more substantial improvements, we then explored alternative classification strategies and new architectures, including a fully connected Feed-Forward Neural Network—that will be referred to as Sequential LSTM—and a Transformer model, the former refining the existing pipeline and the latter providing a fundamentally different perspective for deeper understanding [1]. Towards the end of the project, we observed solid performance from an architecture that combines a 1-dimensional CNN with a basic LSTM [2].

Table 2: Performance Overview over the validation set

Model Variant	F1
LSTM tuned	0.9571
Sequential LSTM	0.9584
Transformer	0.9358
CNN1d + LSTM	0.9642

4.3 Final Model Selection

The CNN1d + LSTM model achieved the best test-set performance:

$$\textbf{F1-score (test)} = \mathbf{0.9495}$$

The Sequential LSTM classifier secured the second-highest test score, achieving an **F1-score** (test) = 0.9440

These models were selected as the final architectures for deployment.

5 Results

Table 2 shows that the standard LSTM, the Sequential LSTM and the CNN1d + LSTM achieve the best validation performance, though only with minor improvements over the baselines. Notably, the simple LSTM performs unexpectedly well despite stronger overfitting, while weighted losses, automatic attention mechanisms and data augmentation offer limited or negative impact on validation results. Although the Transformer displays stable and robust training dynamics, this stability does not translate into better test-set performance.

6 Discussion

The trained models showed a strong tendency to overfit, which neither hyperparameter tuning nor architectural modifications were able to completely resolve. The only approach that mitigated this issue was the Sequential LSTM, likely due to improved gradient flow and generalization. Additionally, using a manually selected subsequence for evaluation proved beneficial.

7 Conclusions

It can be concluded that the strongest contributions to achieving the best test performance stem from the hyperparameter tuning process and the overall effectiveness of the LSTM-based architectures, supported by a consistent and well-designed testing pipeline.

Future improvements may include a more thorough investigation of Transformer-based alternatives as well as a deeper exploration of the Sequential LSTM architecture, particularly with respect to its ability to mitigate overfitting. CNN-including models should be also considered for further analyses.

References

[1] Q. Wen, T. Zhou, C. Zhang, W. Chen, Z. Ma, J. Yan, and L. Sun, *Transformers in Time Series: A Survey*. arXiv:2202.07125, 2023.

[2] Q. Du, W. Gu, L. Zhang, and S.-L. Huang, *Poster Abstract: Attention-based LSTM-CNNs For Time-series Classification*. In *Proceedings of the 16th ACM Conference on Embedded Networked Sensor Systems (SenSys'18)*, ACM, Shenzhen, China, 2018. doi: 10.1145/3274783.3275208.